

Parallel Computing Laboratory

Lab 3: MPI Collective Communication

Huzaifa Arshad

Roll No: 2023-CS-86

Department of Computer Science

Contents

1	Objective	3
2	Introduction	3
3	System Configuration	3
4	Exercise 1: MPI Broadcast	4
4.1	Description	4
4.2	Commands	4
4.3	Output	4
5	Exercise 2: Parallel Sum using MPI Reduce	4
5.1	Description	4
5.2	Commands	4
5.3	Output	5
5.4	Performance Comparison	5
6	Exercise 3: MPI Scatter and Gather	5
6.1	Description	5
6.2	Commands	5
6.3	Output	5
7	Exercise 4: MPI Scatterv and Gatherv	6
7.1	Description	6
7.2	Commands	6
7.3	Output	6
8	Exercise 5: Parallel Matrix Vector Multiplication	6
8.1	Description	6
8.2	Commands	6
8.3	Output	7
9	Self-Assessment Questions	7
9.1	What is MPI?	7
9.2	What is the difference between MPI_Scatter and MPI_Scatterv?	7
9.3	What is the purpose of MPI_Reduce?	7
9.4	What is MPI_Broadcast used for?	7
9.5	Why is parallel computing important?	7
10	Observations	7
11	Conclusion	8

1 Objective

The objective of this laboratory experiment is to understand the Message Passing Interface (MPI) and its role in parallel computing. The lab demonstrates the implementation of different MPI collective communication operations including Broadcast, Reduce, Scatter, Gather, Scatterv, Gatherv, and parallel matrix-vector multiplication.

2 Introduction

Message Passing Interface (MPI) is a standardized and portable message-passing system designed to function on parallel computing architectures. MPI allows multiple processes running on different processors to communicate with each other by sending and receiving messages.

MPI provides two major communication mechanisms:

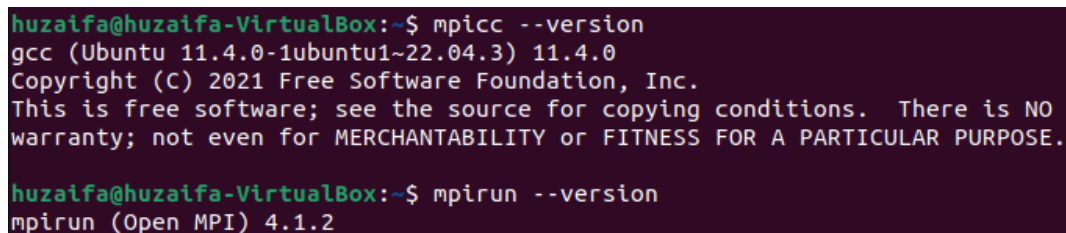
- Point-to-Point Communication
- Collective Communication

Collective communication involves a group of processes participating in data exchange operations. Examples include Broadcast, Reduce, Scatter, and Gather.

In this lab, we explore these collective communication operations and analyze their behavior in a parallel environment.

3 System Configuration

- Operating System: Ubuntu 22.04 (Virtual Machine)
- MPI Library: Open MPI 4.1.2
- Compiler: GCC 11.4.0
- Platform: VirtualBox

A terminal window with a dark purple background and light green text. The prompt is 'huzaiifa@huzaiifa-VirtualBox:~\$'. The first command is 'mpicc --version', which outputs 'gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0' followed by copyright and warranty information. The second command is 'mpirun --version', which outputs 'mpirun (Open MPI) 4.1.2'.

```
huzaiifa@huzaiifa-VirtualBox:~$ mpicc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

huzaiifa@huzaiifa-VirtualBox:~$ mpirun --version
mpirun (Open MPI) 4.1.2
```

Figure 1: MPI Version Information

4 Exercise 1: MPI Broadcast

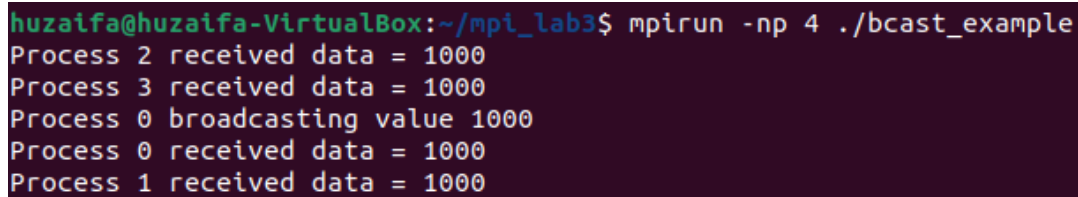
4.1 Description

MPI Broadcast allows a root process to send the same data to all processes within a communicator.

4.2 Commands

```
mpicc -O2 -o bcast_example bcast_example.c
mpirun -np 4 ./bcast_example
```

4.3 Output

A terminal window with a dark purple background and light green text. The prompt is 'huzaiifa@huzaiifa-VirtualBox:~/mpi_lab3\$'. The command entered is 'mpirun -np 4 ./bcast_example'. The output shows five lines: 'Process 2 received data = 1000', 'Process 3 received data = 1000', 'Process 0 broadcasting value 1000', 'Process 0 received data = 1000', and 'Process 1 received data = 1000'.

```
huzaiifa@huzaiifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./bcast_example
Process 2 received data = 1000
Process 3 received data = 1000
Process 0 broadcasting value 1000
Process 0 received data = 1000
Process 1 received data = 1000
```

Figure 2: MPI Broadcast Execution

5 Exercise 2: Parallel Sum using MPI Reduce

5.1 Description

This program calculates the sum of numbers from 1 to 1000 using parallel processes. Each process calculates a partial sum, and MPI_Reduce combines all partial sums to obtain the final result.

5.2 Commands

```
mpicc -O2 -o reduce_sum reduce_sum.c

mpirun -np 1 ./reduce_sum
mpirun -np 2 ./reduce_sum
mpirun -np 4 ./reduce_sum
```

5.3 Output

```
huzaiifa@huzaiifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./reduce_sum
The sum from 1 to 1000 is: 500500
Time with 4 processes: 0.000566 seconds
Expected sum: 500500
```

Figure 3: Parallel Sum using MPI Reduce

5.4 Performance Comparison

Number of Processes	Execution Time (seconds)
1	0.000005
2	0.000062
4	0.000106

6 Exercise 3: MPI Scatter and Gather

6.1 Description

MPI Scatter distributes portions of an array from the root process to all other processes. MPI Gather collects results from all processes and returns them to the root process.

6.2 Commands

```
mpicc -O2 -o scatter_gather scatter_gather.c
mpirun -np 4 ./scatter_gather
```

6.3 Output

```
huzaiifa@huzaiifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./scatter_gather
Process 0: local sum = 31375
Total sum via MPI_Reduce: 500500
Collected sums: 31375 93875 156375 218875
Expected total sum = 500500
Process 1: local sum = 93875
Process 2: local sum = 156375
Process 3: local sum = 218875
```

Figure 4: Scatter and Gather Execution

7 Exercise 4: MPI Scatterv and Gatherv

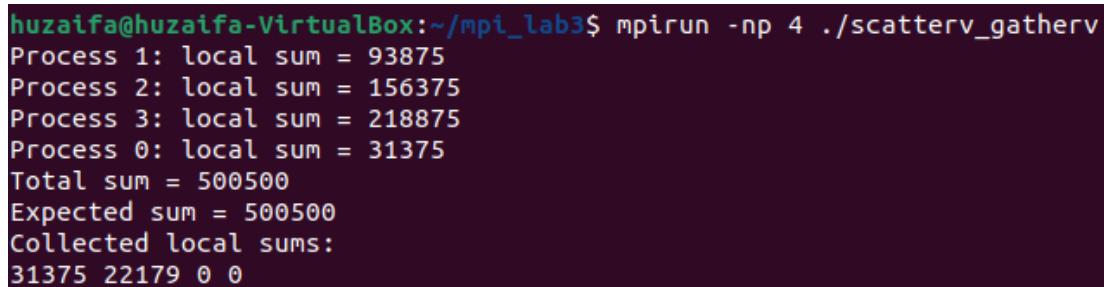
7.1 Description

Scatterv and Gatherv allow processes to send and receive different amounts of data, making them useful for uneven data distributions.

7.2 Commands

```
mpicc -O2 -o scatterv_gatherv scatterv_gatherv.c
mpirun -np 4 ./scatterv_gatherv
```

7.3 Output

A terminal window with a dark purple background and light green text. The prompt is 'huzaifa@huzaifa-VirtualBox:~/mpi_lab3\$'. The command executed is 'mpirun -np 4 ./scatterv_gatherv'. The output shows four processes reporting their local sums, followed by the total sum, expected sum, and collected local sums.

```
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./scatterv_gatherv
Process 1: local sum = 93875
Process 2: local sum = 156375
Process 3: local sum = 218875
Process 0: local sum = 31375
Total sum = 500500
Expected sum = 500500
Collected local sums:
31375 22179 0 0
```

Figure 5: Scatterv and Gatherv Execution

8 Exercise 5: Parallel Matrix Vector Multiplication

8.1 Description

In this exercise, rows of a matrix are distributed among multiple processes. Each process performs partial multiplication with a vector and the results are gathered to form the final output vector.

8.2 Commands

```
mpicc -O2 -o matvec matvec.c
mpirun -np 4 ./matvec
```

8.3 Output

```
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ nano matvec.c
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpicc -O2 -o matvec matvec.c
huzaifa@huzaifa-VirtualBox:~/mpi_lab3$ mpirun -np 4 ./matvec
Result vector y:
8.000000 8.000000 8.000000 8.000000 8.000000 8.000000 8.000000 8.000000
```

Figure 6: Parallel Matrix Vector Multiplication

9 Self-Assessment Questions

9.1 What is MPI?

MPI (Message Passing Interface) is a standardized library used for parallel programming in distributed memory systems. It allows multiple processes to communicate with each other using message passing.

9.2 What is the difference between MPI_Scatter and MPI_Scatterv?

MPI_Scatter distributes equal amounts of data to each process, while MPI_Scatterv allows different amounts of data to be distributed to different processes.

9.3 What is the purpose of MPI_Reduce?

MPI_Reduce combines values from all processes using a specified operation such as sum, maximum, or minimum and returns the result to the root process.

9.4 What is MPI_Broadcast used for?

MPI_Broadcast sends a message from one root process to all other processes in the communicator.

9.5 Why is parallel computing important?

Parallel computing allows large problems to be solved faster by dividing tasks among multiple processors that execute simultaneously.

10 Observations

- MPI enables efficient communication among parallel processes.
- Collective communication operations simplify data distribution and collection.
- All experimental results matched the expected computational values.
- Increasing the number of processes can improve performance depending on the available hardware.

11 Conclusion

This laboratory experiment provided hands-on experience with MPI collective communication operations such as Broadcast, Reduce, Scatter, Gather, Scatterv, and Gatherv. The experiments demonstrated how computational workloads can be distributed among multiple processes to perform parallel computations efficiently.